AXON-RFC-006 Appendix B: easynet.web_app

Technical design, build pipeline, Hub routing, and implementation plan for full-stack agent-deployed applications

Silan Hu
National University of Singapore

Draft v0.1, April 28, 2026

Abstract

This appendix specifies `easynet.web_app`, the state_type that supports a full modern web deployment: a real React/Vue/Next-style frontend project authored by an agent (typically Claude Code) inside its workspace, built into a static `dist/` bundle, and served alongside a backend whose endpoints are not pretend HTTP handlers but genuine EasyNet abilities. The Hub plays the role of a state-aware nginx: it serves the frontend bundle, translates `POST /api/<verb>` requests into ability invocations on the owner agent, proxies WebSocket subscriptions to ability streams, and exposes `/__easynet/agent` as the machine-actionable view of the same application. The appendix covers the modern project model, the canonical / runtime state split, the long-running build pipeline, the API ability mapping, the Hub routing matrix (with worked CSP and caching rules suitable for SPAs), the agent_view shape that makes the entire application discoverable to peer agents, the five PR slices required to land it in `EasyNet-Cli`, and an end-to-end worked example of an agent authoring a Vite + React shop, declaring its API abilities, building, publishing, serving, and being audited.

Contents

1 Position relative to pages and to the network history of the web

[derived] The web evolved from a static-document medium into a full application platform. EasyNet's stateful web layer follows the same arc, in two appendices:

Appendix	What it models
A (`easynet.page`)	A single document. Author writes HTML / Markdown; Hub serves it. The minimal reference — one canonical state, no runtime overlay, no build step, no API surface.
B (`easynet.web_app`)	A full application. Author writes a real frontend project, a build pipeline produces a static bundle, the application's API endpoints ARE EasyNet abilities. Hub serves frontend + API + WebSocket + agent_view from a single URL space.

We keep both. Pages remains the simplest verification of the protocol's normative core; web_app is the first state type that actually exercises the full pressure-test surface (§B.1–B.8 of the markdown companion). An implementation that ships pages but not web_app has solved the easy half of stateful EasyNet; this appendix specifies the hard half.

Why this is not nginx. Nginx serves a static directory and reverse-proxies to a separately-

deployed backend. The backend is a black box — nginx neither understands its state, nor knows what endpoints it offers, nor can answer "what can a caller do here" to a peer asking through structured discovery. The Hub specified here serves the same kinds of bytes nginx would, but its routing decisions are derived from the state object's canonical state, its API endpoints are abilities introspectable through the same registry that backs every other EasyNet caller, and its dual-view contract means the same URL space carries both human content and an agent-actionable map of the entire application. Nginx is HTTP plumbing; the Hub is a state-projection layer that happens to speak HTTP at its outer boundary.

2 Goals and non-goals

2.1 Goals

[normative] An implementation conforming to this appendix:

1. Lets an agent author a real frontend project in its workspace using whatever toolchain the agent prefers (Vite, Next.js, Remix, etc.); EasyNet does not invent a project structure.
2. Provides a long-running canonical transition, `webapp.build`, that runs the project's declared build command, captures the resulting `dist/` (or equivalent) as an immutable content-addressed bundle, and advances canonical state on success or failure.
3. Provides runtime overlay transitions (`webapp.serve`, `webapp.stop`) that bind a local port and start serving — crucially, without changing canonical state.
4. Translates HTTP requests on the Hub into EasyNet operations along three orthogonal channels: static-bundle fetch, API ability invocation, and WebSocket ability subscription.
5. Exposes a single `agent_view` URL (`/__easynet/agent`) that returns the application's full discoverability surface: routes, API endpoints, `ability_surface` for transitions, build metadata, receipt history pointer, and an EAL hint.
6. Implements all of the above in five PR slices (§9), each independently reviewable.

2.2 Non-goals

[derived] Out of scope for this appendix:

- **TLS termination, HTTP/2 negotiation, multi-instance Hub clusters.** Operations work, not protocol work.
- **Choice of frontend framework.** The appendix references React/Vite as the worked example because Claude Code defaults to it; the protocol is framework-agnostic.
- **Server-side rendering with persistent process.** Phase 1 supports static + Hub-routed API + WebSocket. SSR that needs a Node.js process running per request is RFC-006-B v2.
- **Cross-realm or cross-node application federation.** A web_app is owned by exactly one agent on exactly one node in Phase 1.
- **Authenticated humans (`principal/human-auth`).** Phase 1 supports `principal/human-anon` only; cookies set by the application itself are passed through, but EasyNet does not issue them.

3 Modern project model

3.1 What lives in the workspace

[normative] The state object's source-of-truth on disk is *a normal project directory*, owned by the agent. Concretely:

```
<workspace>/web_apps/<slug>/
   easynet.app.toml          # the EasyNet-side declaration
   package.json              # owned by the agent's toolchain
   package-lock.json         # ditto
   src/                      # framework-shaped source
   public/
   vite.config.ts            # or next.config.js, etc.
   (optionally) anything else the framework needs
```

EasyNet **owns** only `easynet.app.toml`; the rest is the framework's. When the agent runs `webapp.build`, the build command declared in `easynet.app.toml` executes in this directory; whatever it produces in the declared `dist_dir` is captured as the build artifact.

3.2 easynet.app.toml

[normative] The single file EasyNet reads to understand a web_app:

```
# easynet.app.toml — the only file EasyNet authors and reads.
# The rest of the project belongs to the framework.

[manifest]
name         = "shop"
description  = "Tiny ceramic shop, agent-built."
visibility   = "public"            # private | realm | public

[build]
command      = "npm install && npm run build"
dist_dir     = "dist"              # relative to project root
build_timeout_secs = 600
node_version = "20"                # advisory; executor may pin
                                   # via a stable images registry

[serve]
# When `webapp.serve` is invoked the runtime binds a port and
# starts a process whose only job is to expose API+WS for the
# Hub's reverse-proxy use; the static bundle is served from the
# blob store, NOT from this process.
mode         = "static-only"
               # "static-only"          : no backend process; API
               #                          is satisfied entirely by
               #                          ability invocations from
               #                          the Hub. The default and
               #                          the Phase 1 production
               #                          shape.
               # "process"              : a backend process is
               #                          started by the executor;
               #                          its bound port appears in
               #                          runtime overlay state.
               #                          Reserved for v2.

[api]
```

```
# Abilities the FRONTEND is allowed to call through the Hub's
# /api/<verb> route. Each entry is a fully-qualified ability name
# the owner agent has registered. The Hub will refuse to translate
# a /api/<verb> request whose verb is not on this list; this is
# the explicit attack surface the agent has authorised.
abilities = [
  "shop.list_items",
  "shop.checkout",
  "shop.order_status"
]

[routes]
# nginx-shaped routing rules. Order is significant.
# Each rule has a `match` pattern and an `action`.
rules = [
  { match = "exact:/",                      action = "static:dist/index.html" },
  { match = "prefix:/assets/",              action = "static:dist/assets/" },
  { match = "regex:^/api/([a-z][a-z0-9_]*)$",
    action = "api:$1",
    methods = ["GET","POST"] },
  { match = "prefix:/ws/",                  action = "ws:$path" },
  { match = "exact:/__easynet/agent",       action = "agent_view" },
  { match = "exact:/__easynet/health",      action = "health" },
  { match = "exact:/__easynet/receipts",    action = "receipts" },
  { match = "fallback",                     action = "spa:dist/index.html" }
]

[csp]
# Tighten or loosen the default. The default is suitable for a
# vanilla SPA; frameworks that need 'unsafe-inline' must opt in.
extra_script_src = []   # e.g. ["'sha256-...'"] for inline bootstrap
extra_style_src  = ["'unsafe-inline'"]
extra_connect_src= ["self"]
```

The file is small, declarative, and the only thing whose hash participates in canonical state alongside the source-tree and build-artifact hashes. A minimal app gets by with three sections (`[manifest]`, `[build]`, `[api]`); routes default to the standard SPA layout shown above.

3.3 Why declared API surface, not auto-detected

[normative] A web_app's API surface is declared, not discovered. The Hub will refuse to translate `/api/<verb>` when `<verb>` is not in `[api].abilities`. Two reasons:

- **Audit clarity.** The receipt log records exactly which abilities a public web_app exposes. A peer auditing the application sees the surface in canonical state, not as a runtime introspection.
- **Attack surface containment.** The owner agent might host many abilities; only those explicitly listed are reachable through the public Hub. An ability the agent registered for internal use is invisible from the web.

Invariant — WB-INV-1

The Hub **MUST** refuse to translate `/api/<verb>` into an ability invocation unless `<verb>` appears in the web_app's currently-Built canonical state's `api_abilities` list. The refusal MUST be HTTP 404 (not 403) to avoid confirming the existence of internal abilities. This is the application-layer instance of the trust-boundary discipline TR-INV-12 establishes at the identity layer.

4 State machine

4.1 state_type and state_key

[normative]

```
state_type:  easynet.web_app

state_key:
    (realm: string, owner_agent_uri: string, app_slug: string)

  realm           Logical namespace; same conventions as pages.
  owner_agent_uri Canonical authority. The agent that signs every
                  CanonicalReceipt for this app. Embedded in the
                  key (not a field) per main §1.3.
  app_slug        Owner-namespaced kebab-case identifier
                  (^[a-z0-9][a-z0-9-]{0,62}[a-z0-9]$).

URA form:
  easynet:///r/<realm>/agent/<owner>/web_app/<slug>
```

4.2 Canonical fields

[normative] Per CSH-INV-2 (main §2.5) every large value is held by content hash:

```
canonical_fields(web_app) = {
    manifest_hash:       bytes(32),  // SHA-256 of canonicalised
                                     // easynet.app.toml's [manifest]
                                     // section (name, description,
                                     // visibility)
    build_config_hash:   bytes(32),  // canonicalised [build]
                                     // section: command, dist_dir,
                                     // build_timeout, node_version
    api_surface_hash:    bytes(32),  // canonicalised
                                     // [api].abilities (sorted)
    routes_manifest_hash:bytes(32),  // canonicalised [routes].rules
                                     // (in declared order)
    csp_hash:            bytes(32),  // canonicalised [csp] section
    source_snapshot_hash:bytes(32),  // hash of the source tree at
                                     // the most recent build. Null
                                     // before first build attempt.
    build_artifact_hash: bytes(32),  // hash of the dist tree
                                     // produced by the last
                                     // successful build. Null until
                                     // first Built. Cleared on
                                     // delete.
    state_value:         string,     // §4.3
    visibility:          string,     // mirrors manifest.visibility
```

```
                                    // for fast scope check
    version:             uint64
}
```

Canonicality rule. Every field above is tagged `canonical:true` in the schema; the schema-driven projection π produces the projection over exactly these keys.

4.3 Canonical state space

state_value	Meaning
`Created`	easynet.app.toml registered; no build attempted yet.
`Building`	A `webapp.build` is in flight. Transient. Visible only via OperationalReceipts; never the terminal canonical state.
`Built`	At least one successful build exists. `build_artifact_hash` non-null. Web-app may be Served.
`Failed`	The most recent build failed. Owner can iterate and retry.
`Deleted`	Terminal; `build_artifact_hash` cleared, blob ref-count decremented.

4.4 Runtime overlay state space

runtime	Meaning
`Stopped`	No serving overlay associated with this app.
`Starting`	`webapp.serve` invoked; Hub is binding the slug into its router.
`Running`	Hub is actively translating `/<slug>/...` requests for this app.
`Crashed`	The Hub lost its mapping (process restart, config-reload error, etc.); canonical state untouched. Recoverable via `webapp.serve`.

The canonical $\times$ runtime cross-product matrix is identical in structure to the one shown in §B.4.3 of the markdown companion: only `Built` admits non-Stopped runtime overlays.

4.5 Transition list

Transition	Class	Notes
`webapp.create@v1`	canonical	∅ → Created. Mints state_key, persists manifest/build_config/routes/api/csp hashes.
`webapp.update_config@v1`	canonical	Created/Built/Failed → same; replaces declared config (manifest, build, api, routes, csp). Does NOT trigger a build.
`webapp.build@v1`	canonical	Created/Built/Failed → Building (transient) → Built|Failed. Long-running. See §5.
`webapp.delete@v1`	canonical	* → Deleted. Implicit `webapp.stop` as side effect.
`webapp.serve@v1`	operational	Stopped/Crashed → Starting → Running. Requires canonical = Built.
`webapp.stop@v1`	operational	Starting/Running/Crashed → Stopped.
`webapp.get@v1`	query	Returns canonical + runtime snapshot.
`webapp.list@v1`	query	Lists web_apps owned by an agent.

5 Build pipeline

5.1 Steps of webapp.build@v1

[normative] On `webapp.build` the executor:

1. Reads `easynet.app.toml`, validates it.
2. Computes `source_snapshot_hash`: a Merkle hash over the project directory (excluding `node_modules/`, `dist/`, `.git/`, and any path the framework's `.gitignore` excludes). The hash is over a sorted list of `(relative_path, sha256(file_bytes))` pairs, JCS-canonicalised.
3. Emits `build.started` *OperationalReceipt* with the source-snapshot hash, declared command, expected timeout. *Not canonical.*
4. Spawns the build command in the project directory. Stdout / stderr are tee'd to a streaming log; periodic `build.progress` OperationalReceipts are emitted with a hash of the cumulative log. Lossy.
5. On exit:
   - Status 0 + `dist_dir` non-empty: walk the `dist_dir` tree; compute `build_artifact_hash` as a Merkle hash over `(relative_path, sha256(file_bytes))` pairs; write the entire tree to the blob store as one `TreeBlob` (a JCS-canonical manifest of file paths and individual blob hashes; the file blobs are stored as ordinary content-addressed blobs).
   - Status non-zero or empty `dist_dir`: prepare a failure-reason payload (last 64KB of log + exit status) and store its hash.
6. Emit one *CanonicalReceipt*: `build.completed` (success) or `build.failed` (failure). This is the sole canonical advance for the transition. `post_state_hash` reflects Built or Failed; `state_version` +1.

Canonicality discipline. The transient `Building` state value, the streaming progress, and the build-tool stdout are all observability. They never enter the canonical hash. A verifier given only canonical receipts sees the application transition Created → Built (or Failed) in a single step — which is correct: the canonical fact is the artifact, not the journey.

5.2 TreeBlob format

[normative] A TreeBlob is the content-addressed representation of a directory tree. It comprises:

```
TreeBlob (JCS-canonical JSON)
{
  "version": 1,
  "entries": [
    { "path": "index.html",
      "size": 2148,
      "blob": "<sha256-of-file-bytes>",
      "mode": "0644" },
    { "path": "assets/index-DfX9k.js",
      "size": 187321,
      "blob": "<sha256>",
      "mode": "0644" },
    ...
  ]
}
```

The tree's hash is `sha256(jcs(TreeBlob))`; this is `build_artifact_hash`. Each individual file is also a content-addressed blob. The Hub fetches a file by computing `path lookup` in the TreeBlob and then `blob.get(<file-blob-hash>)`; this is one indirection but lets the blob store dedupe identical asset bytes across builds and across applications.

Why not git tree hash directly. Git's tree hash is also content-addressed but bakes mode bits and entry types in a git-specific way. We pick a JCS-flat format so the hash function is the same as everywhere else in EasyNet (SHA-256 over JCS bytes), avoiding a second canonicalisation regime.

5.3 Build executor

[normative] Phase 1 ships a *local* build executor: the build runs in a subprocess of the EasyNet daemon owning the agent. The owner signs the resulting CanonicalReceipt directly. Delegated build (the source lives on agent A, build runs on agent B's hardware) is the future: the protocol is already shaped for it (main §5.3) but the executor is not.

Open decision

Open: build sandboxing. The build subprocess has the EasyNet daemon's process privileges. A malicious build script could read the daemon's other agent state. Phase 1 mitigation: the build runs in the project directory with a restricted `PATH`, no inherited env vars except `NODE_VERSION` and `HOME`, and a stricter `ulimit`. Full sandboxing (seccomp / pledge / Docker) is RFC-006-B v2.

6 Hub routing model

The Hub is bound to a single port (Phase 1: `127.0.0.1:8787` by default). Every incoming HTTP request is processed in seven steps:

1. **Caller minting.** Build envelope per main §5.4: `caller = principal/human-anon/<ip-hash>/<sid>`; `caller_context` per main §3.6 / TR-INV-13.
2. **Path parsing.** Path is one of:
   - `/<realm>/<owner>/<slug>/...` — canonical form.
   - Any path under a configured alias domain (e.g. `shop.alice.example` mapping to one specific (`realm, owner, slug`) tuple).
3. **State resolve.** Look up the web_app in the state store. If not Built, return per §6.1. If `visibility` forbids the caller, return 404.
4. **Route match.** Run the path tail through the web_app's `[routes].rules`, in declared order, until one matches.
5. **Action dispatch.** Dispatch on the matching rule's action (static / api / ws / agent_view / health / receipts / spa fallback). See §6.2.
6. **Response build.** Set CSP, ETag, cache headers, content-type per action.
7. **OperationalReceipt.** Emit one OperationalReceipt with caller, caller_context, matched rule, action, response status. *Not canonical.*

6.1 Hub behavior matrix per state

Canonical	Runtime	Hub response
`Created`	*	503 "not built". `Retry-After: 60`.
`Building`	*	503 + small progress page that streams build.progress receipts via SSE.
`Built`	`Stopped`	503 "not serving".
`Built`	`Starting`	503 "starting", `Retry-After: 2`.
`Built`	`Running`	Full routing matrix below.
`Built`	`Crashed`	503 "recoverable failure" + reason hash; `Retry-After: 5`.
`Failed`	*	503 "last build failed" + reason hash.
`Deleted`	*	404.

6.2 Action dispatch

static:<path> Resolve `<path>` inside the web_app's `build_artifact_hash` TreeBlob. Fetch the file blob. Reply with:

```
HTTP/1.1 200 OK
Content-Type: <inferred from extension; allow-list mapping>
Content-Length: <size>
ETag: "<base64url(blob_hash[..16])>"
Cache-Control: public, max-age=31536000, immutable
                                # only for /assets/* (hashed names)
                                # otherwise: max-age=60, must-revalidate
```

```
Content-Security-Policy: <built from app's csp_hash>
Referrer-Policy: no-referrer
X-Content-Type-Options: nosniff
```

The `immutable` cache policy is safe for hashed asset filenames (Vite/Webpack output a content hash in the filename itself); `index.html` and other un-hashed files use a short `must-revalidate` window.

api:<verb> `<verb>` is the regex capture from the route rule. The Hub:

1. Validates that `<owner>.<verb>` is in `api_surface_hash`'s expanded ability list. Refusal: HTTP 404 (WB-INV-1).
2. Reads request body (size-capped at 1 MiB Phase 1) as JSON. The JSON is the ability args directly — no envelope wrapping. `GET` requests use query string; the Hub parses query string into an object.
3. Builds an EasyNet invocation with this caller, caller_context, and the parsed args.
4. Invokes `<owner>.<verb>`.
5. Maps result to HTTP:
   - `Succeeded`: 200, body = receipt's result body, content-type `application/json`.
   - `Failed` with a structured error ability receipt: 400 if validation, 500 otherwise; body = `{ error_code, error_message, receipt_id }`.
   - Timeout: 504; receipt_id of the operational timeout receipt.

Idempotency. Both `GET` and `POST` are allowed; idempotency is the ability's responsibility, not the Hub's. The Hub does NOT enforce that GET produces no CanonicalReceipt; it is an EasyNet ability and may emit one if that's its intended semantics. The frontend chooses the verb; ability authors document idempotency in their schema.

ws:<path> The Hub upgrades the connection to WebSocket and binds it to an ability *stream* (the existing EasyNet stream subscription mechanism). The first WebSocket message from the client MUST be a JSON `{ "ability": "<verb>", "args": {...} }`; the Hub then opens an `<owner>.<verb>` stream and forwards each emitted event as a WebSocket text frame. Client messages after the first are forwarded back to the stream's input channel. WebSocket close maps to stream close.

agent_view Returns the JSON specified in §7.

health Returns `200 {"state":"...","runtime":"..."}`.

receipts Returns the canonical-receipt history for this state_key, optionally bounded by `?from=<version>&to=<version>`.

spa:<fallback-path> Used as the last `fallback` rule. Identical to `static:<fallback-path>` but with `Cache-Control: no-cache` so the SPA shell is always fresh while the hashed assets it loads are immutable.

7 agent_view: full-app discoverability

[normative] The `agent_view` of a Built/Running web_app:

```
GET /<realm>/<owner>/<slug>/__easynet/agent

200 OK
Content-Type: application/json

{
  "state_type":  "easynet.web_app",
  "state_key":   { "realm":"default","owner":"alice","slug":"shop" },
  "ura":         "easynet:///r/default/agent/alice/web_app/shop",

  "canonical": {
    "state":              "Built",
    "manifest":           { "name":"shop",
                            "description":"Tiny ceramic shop, agent-built.",
                            "visibility":"public" },
    "build_artifact_hash":"0x4d28ee17...",
    "version":            7,
    "etag":               "TSjuFw0pAvjpAQ=="
  },

  "runtime": {
    "state":     "Running",
    "started_at":"2026-04-28T09:11:02Z",
    "host":      "https://shop.alice.example"
                 // null if no alias domain is configured
  },

  "ability_surface": [   // transitions admissible RIGHT NOW
    "webapp.update_config@v1",
    "webapp.build@v1",
    "webapp.stop@v1",
    "webapp.delete@v1"
  ],

  "api": {               // what the FRONTEND can call via /api/*
    "endpoints": [
      { "verb":"shop.list_items",
        "http": "GET  /api/list_items",
        "schema_url": "/__easynet/schema/shop.list_items" },
      { "verb":"shop.checkout",
        "http": "POST /api/checkout",
        "schema_url": "/__easynet/schema/shop.checkout" },
      { "verb":"shop.order_status",
        "http": "GET  /api/order_status",
        "schema_url": "/__easynet/schema/shop.order_status" }
    ]
  },

  "routes": [
    { "match":"exact:/",          "action":"static:dist/index.html" },
    { "match":"prefix:/assets/", "action":"static:dist/assets/" },
    { "match":"regex:^/api/...","action":"api:$1" },
    { "match":"prefix:/ws/",     "action":"ws:$path" },
    { "match":"fallback",        "action":"spa:dist/index.html" }
```

```
  ],

  "build_meta": {
    "command":          "npm install && npm run build",
    "dist_dir":         "dist",
    "node_version":     "20",
    "last_build_at":    "2026-04-28T09:09:51Z",
    "build_duration_s": 38.2,
    "framework_hint":   "vite-react"   // best-effort sniff
  },

  "history": {
    "versions":                    7,
    "latest_canonical_receipt_id": "r_c3d4...",
    "receipt_log_ura":
       "easynet:///r/default/agent/alice/web_app/shop/receipts"
  },

  "eal_hint":
    "call shop.checkout on alice with { item_id: \"mug-12oz\", qty: 1 } timeout 30"
}
```

Why this is the killer feature. A peer agent reading this URL gets, in one document, everything it needs to:

- know what application this is and what state it's in;
- know exactly which API verbs it can call (with paths, methods, and schema pointers);
- know which transitions advance the application (`ability_surface`);
- invoke any of those verbs directly via EasyNet, OR via the public `/api/<verb>` endpoint, since both paths reach the same ability;
- replay the application's history end-to-end and verify cryptographic chain back to the owner.

This is the agent-native web. A traditional web app exposes its HTML to humans and its API to documented endpoints; a peer program needs the documentation, an OpenAPI spec, an SDK, and trust. This view collapses all four.

8 Architecture

8.1 Module map (EasyNet-Cli)

```
src/runtime/state/                    [shared with pages, P-A1]

src/runtime/blob_store/               [shared with pages, P-A1]
  tree.rs                             ← NEW: TreeBlob put/get,
                                       per-path lookup

src/runtime/agents/webapp_ability.rs  ← NEW (P-B2)
                                      register: create / build /
                                      update_config / serve / stop /
                                      delete / get / list

src/runtime/build_executor/           ← NEW (P-B1)
```

```
  mod.rs                                long-running canonical
                                        transition driver
  source_snapshot.rs                    Merkle hash of project dir
  spawn.rs                              subprocess + log streaming
  artifact_capture.rs                   dist tree -> TreeBlob

src/runtime/views/webapp/               ← NEW (P-B5)
  agent_view.rs                         JSON projection of §6
  health.rs
  receipts.rs

src/runtime/hub/                        [extends pages Hub from P-A4]
  router.rs                             ← extended for routes_manifest
  static_handler.rs                     ← NEW (P-B3): TreeBlob fetch
  api_handler.rs                        ← NEW (P-B3): /api/<verb>
                                         translation
  ws_handler.rs                         ← NEW (P-B4): /ws/* upgrade
                                         and stream binding
```

8.2 Daemon wiring

The daemon at boot adds, on top of the pages wiring:

1. Open the build executor's working directory at `$EASYNET_HOME/build-cache/`.
2. Register `webapp_ability` for each agent.
3. For every `web_app` whose canonical state is `Building` (i.e. a daemon-restart interrupted a build): mark Failed with reason "daemon restart during build". (TR-INV-1's discipline: only terminal canonical receipts advance state; an interrupted build never reached its terminal, so canonical state is rolled back to whatever preceded Building — `Created` or `Failed`. The reset to Failed is a *new* transition, not a rollback.)

8.3 Hub wiring

The Hub binary, `easynet-hub`, gains in P-B3/P-B4 the `static_handler` / `api_handler` / `ws_handler` alongside the page handler from P-A4. Its top-level dispatch is:

```
fn handle(req: HttpRequest) -> HttpResponse {
    let envelope = build_envelope(&req);                  // §5.7 PA
    let (realm, owner, slug, tail) = parse_path(&req.uri)?;
    let kind = state_store.classify(realm, owner, slug)?; // page or web_app

    match kind {
        Kind::Page    => pages_dispatch(envelope, owner, slug, tail),
        Kind::WebApp  => webapp_dispatch(envelope, owner, slug, tail),
        Kind::Unknown => http_404(),
    }
}
```

The Hub is single-binary even when serving both pages and web_apps. The path scheme distinguishes them only by what's registered in the state store under `(owner, slug)` — the URI form is identical because both are state objects under the same agent's namespace.

9 Implementation plan

Five PR slices, sequenceable. Slice P-B1 builds on P-A1 (the blob store and state store skeleton from the pages appendix); P-B3/B4 build on P-A4 (the basic Hub binary).

9.1 P-B1: build executor + TreeBlob

New files. `src/runtime/build_executor/` (4 modules), `src/runtime/blob_store/tree.rs`.

Public API.

```
pub trait BuildExecutor: Send + Sync {
    fn execute(
        &self,
        project_dir: &Path,
        config: &BuildConfig,
        progress: ProgressSink,    // SSE channel to OperationalReceipts
    ) -> Result<BuildOutcome>;
}
pub enum BuildOutcome {
    Success { artifact_hash: [u8; 32], tree_blob: TreeBlob, log_blob: [u8;32] },
    Failure { reason_hash:   [u8; 32], log_blob:   [u8; 32], exit_status: i32 },
}

pub fn source_snapshot_hash(project_dir: &Path, ignore: &[Pattern])
    -> Result<[u8; 32]>;

pub fn capture_dist(dist_dir: &Path, blob: &dyn BlobStore)
    -> Result<(TreeBlob, [u8; 32])>;
```

Tests.

- Snapshot determinism: two snapshots of the same tree yield identical hashes regardless of file walk order.
- Build smoke: a tiny shell-script "builder" (`mkdir dist && echo hi > dist/index.html`) drives the executor end-to-end; assert TreeBlob round-trips and the artifact hash recomputes deterministically.
- Daemon-restart-during-build: launch a build that sleeps; SIGKILL the daemon; on restart, assert the web_app's canonical state has been reset to Failed with the documented reason.

9.2 P-B2: webapp.* abilities

New files. `src/runtime/agents/webapp_ability.rs`.

Behaviour. Every transition in §4.5 is implemented, including the long-running build (which streams progress OperationalReceipts and emits exactly one terminal CanonicalReceipt per attempt, per TR-INV-1).

Tests.

- Round-trip create/update_config/build/serve/stop/delete.
- Reject `webapp.serve` from `Created` (canonical precondition: Built).
- Reject `api/<verb>` translation when `<verb>` not in `api_surface`.

- Optimistic concurrency: two `webapp.update_config` attempts racing on one web_app; one wins, one rejected with ETag mismatch.
- Build progress receipts are loseable: drop a random subset; replay still reaches the same terminal canonical state.

9.3 P-B3: Hub static + API routing

New files. `src/runtime/hub/static_handler.rs`, `src/runtime/hub/api_handler.rs`; extend `src/runtime/hub/router.rs` to consult `routes_manifest`.

Behaviour.

1. Static handler: TreeBlob path lookup, blob fetch, extension-based MIME detection (allowlist), CSP from `csp_hash`, immutable cache for `/assets/*`.
2. API handler: route regex captures the verb, validate against `api_surface`, parse JSON body or query string into args, invoke `<owner>.<verb>`, map result.

Tests.

- Integration: spin up daemon + Hub in a tempdir, build a fixture web_app whose `api`.abilities contains a registered `shop.echo` that returns its args; `POST /api/echo {"x":1}` yields `200 {"x":1}`.
- Unauthorised verb: `POST /api/secret` returns 404 even when `<owner>.secret` is registered (because not in `api_surface`).
- Static immutable: GET an `/assets/` path twice with `If-None-Match`; second response is 304.

9.4 P-B4: WebSocket proxy + SPA fallback

New files. `src/runtime/hub/ws_handler.rs`.

Behaviour.

1. WS upgrade on `/ws/*`; first text message must be a `{ability,args}` JSON; bind to `ability stream`; forward subsequent text frames.
2. SPA fallback: if no `static` or other rule matches, serve `dist/index.html` with `no-cache`.

Tests. WS streaming: a fixture `shop.tick` streams `{tick:N}` every 100ms; client subscribes via `/ws/tick`, asserts 5 frames received within 600ms; client closes WS, daemon's stream subscription closes within 500ms.

9.5 P-B5: agent_view + meta endpoints

New files. `src/runtime/views/webapp/` (3 modules); minor router extensions for the `/__easynet/*` prefix.

Behaviour.

1. `/__easynet/agent` returns the JSON of §6.
2. `/__easynet/health` returns canonical and runtime states only.
3. `/__easynet/receipts` returns canonical receipts in version order, bounded by query params.
4. `/__easynet/schema/<verb>` returns the ability's input/output schemas as JSON.

Tests.

- Replay: client fetches `/__easynet/receipts`, reconstructs canonical state per receipt, and computes `canonical_state_hash` matching the agent_view's `etag`.
- Schema: agent_view's `api.endpoints[*].schema_url` resolves to a real JSON schema, and that schema admits args that the API handler does not reject.

10 End-to-end worked example

10.1 Setup

```
$ easynet agent add alice --type claude-code
$ easynet-daemon &
$ easynet-hub --bind 127.0.0.1:8787 &
```

10.2 Author the project

Claude Code, running as alice, in alice's workspace:

```
$ cd $EASYNET_HOME/workspaces/alice
$ npm create vite@latest web_apps/shop -- --template react-ts
$ cd web_apps/shop
$ # ... write src/App.tsx with a product list and checkout button
$ # ... write easynet.app.toml as in §3.2
```

It also registers the API abilities through the EasyNet ability scaffolder (out of scope for this appendix; see Appendix "ability authoring"):

```
$ easynet ability new shop.list_items --description "list items" --lang sh
$ easynet ability new shop.checkout   --description "place an order" --lang sh
$ easynet ability deploy abilities/shop.list_items --node alice
$ easynet ability deploy abilities/shop.checkout   --node alice
```

10.3 Create, build, serve

```
$ easynet ability invoke alice.webapp.create --args '{
    "slug":"shop",
    "config_dir":"web_apps/shop"
}'
{ "state":"Created", "version":1, "canonical_state_hash":"0xa3..." }

$ easynet ability invoke alice.webapp.build --args '{ "slug":"shop" }' \
   --stream
[build.started   ] command="npm install && npm run build"
[build.progress  ] log_hash=0x12... 8% installed
[build.progress  ] log_hash=0x13... 47% bundled
[build.progress  ] log_hash=0x14... 92% optimised
[build.completed ] artifact_hash=0x4d28ee17... duration=38.2s
{ "state":"Built", "version":2,
  "build_artifact_hash":"0x4d28ee17..." }

$ easynet ability invoke alice.webapp.serve --args '{ "slug":"shop" }'
{ "runtime_state":"Running" }
```

10.4 Browser fetch

```
$ curl -i http://127.0.0.1:8787/default/alice/shop/

HTTP/1.1 200 OK
Content-Type: text/html
ETag: "TSjuFw0pAvjpAQ=="
Cache-Control: max-age=60, must-revalidate
Content-Security-Policy: default-src 'self'; script-src 'self'; ...
Content-Length: 1247

<!doctype html><html><head>...<script type="module" src="/assets/index-DfX9k.js">...
```

The browser then fetches `/assets/index-DfX9k.js` — `public, max-age=31536000, immutable`, served from blob store. The SPA renders, lists products, the user clicks Buy.

10.5 API call from the browser

```
POST /default/alice/shop/api/checkout HTTP/1.1
Content-Type: application/json
Cookie: easynet_session=sess-abc

{ "item_id":"mug-12oz", "qty":1 }


HTTP/1.1 200 OK
Content-Type: application/json

{ "order_id":"ord_77a3f", "eta":"2026-04-29" }
```

The Hub:

1. Mints `caller=easynet:///principal/human-anon/9c4e.../sess-abc`.
2. Validates `checkout` is in `api_surface` (yes).
3. Invokes `alice.shop.checkout` with the parsed body as args.
4. Returns the receipt's result body to the browser.

10.6 Peer agent discovers the application

```
$ curl http://127.0.0.1:8787/default/alice/shop/__easynet/agent | jq .api
{
  "endpoints": [
    { "verb":"shop.list_items","http":"GET  /api/list_items",
      "schema_url":"/__easynet/schema/shop.list_items" },
    { "verb":"shop.checkout","http":"POST /api/checkout",
      "schema_url":"/__easynet/schema/shop.checkout" }
  ]
}
```

A peer agent — say bob, running on a different node — fetches this view, reads the schema for `shop.checkout`, and invokes the ability either through the EasyNet bus (cross-node) or through the public `/api/checkout` (same Hub). Both paths reach the same ability handler and produce the same kind of canonical receipt.

10.7 Audit replay

The `shop` web_app's history shows three CanonicalReceipts (create, build, build): each carries a `post_state_hash` and an `owner_signature`. The browser-driven checkout emitted a CanonicalReceipt for an entirely different state_type (an `order` state object owned by alice or alice's order service); that receipt also chains back to alice's signing key. A peer reading both chains can reconstruct, byte-for-byte:

- which build artifact the page was at when the checkout happened;
- which API endpoints were declared at that build;
- that the checkout request reached `shop.checkout` version K and produced order `ord_77a3f`;
- that all of the above was signed by alice.

This is the audit story that decisively cannot be told on the traditional web. A CDN serving the shop today, an A/B testing rewrite tomorrow, and an out-of-band order processor on the third day fragment the truth. The web_app collapses it into a single signed receipt chain.

11 Open questions deferred to RFC-006-B v2

1. **Server-side rendering with persistent process.** Phase 1 supports `serve.mode = static-only`. Frameworks that need a Node.js process per request (Next.js with SSR, Remix loaders) need `serve.mode = process`: the executor binds a port, runs the process, the Hub reverse-proxies. The runtime overlay state then carries `bound_port`, `process_id`, `health_endpoint`; a process crash transitions runtime to `Crashed` without touching canonical (TR-INV-5 already requires this).
2. **Cross-node hubs.** A web_app owned by node A served by a Hub on node B. The Hub becomes a federation client; the API handler invokes `<owner>.<verb>` via federation rather than locally.
3. **Signed asset bundles for SRI.** The TreeBlob already provides a hash per file. Embedding those hashes as Subresource Integrity attributes in `index.html` is a straightforward post-build pass; Phase 1 leaves the SRI pipeline to the framework's own plugins.
4. **Build sandboxing.** See the Open decision in §5.
5. **Multi-version routing.** A user wants `/v7/...` to serve build version 7 while `/...` serves the latest. Today the canonical state is single-version per slug. v2 might introduce `webapp.tag` transitions associating versions with routable aliases.

11.1 Coverage check

Main rule	Discharged by	Where
SO-INV-1 (key immutability)	state_key parser refuses key updates	§4
CSH-INV-1 (hash over projection)	schema-driven π, JCS, SHA-256	§4
CSH-INV-2 (content as blob)	TreeBlob + per-file blobs; canonical state holds only hashes	§5
RC-INV-1 (receipt binding)	every webapp receipt carries (state_key, transition_id, attempt_id)	§4
RC-INV-2 (optimistic concurrency)	ETag check on update_config	§9 P-B2
TA-INV-1 (transition_id required)	every `webapp.<verb>@v1` validated	§4
VP-INV-1 (ability_surface from state)	`ability_surface` derived from state_value	§7
TR-INV-1 (long-running terminal)	build emits exactly one terminal CanonicalReceipt	§5
TR-INV-2 (progress not canonical)	`build.progress` are OperationalReceipts	§5
TR-INV-5 (runtime overlay independent)	runtime crash leaves canonical untouched	§4
TR-INV-7 (multi-view first-class)	human (HTML+API+WS) and agent_view both implemented	§6, §7
TR-INV-8 (cross-state-space matrix)	canonical × runtime admissibility enforced	§4
TR-INV-11 (URA-receipt traceability)	`/__easynet/receipts` + state_key index	§10
TR-INV-12 (principal/human-anon)	Hub mints the URI on every request	§6
TR-INV-13 (caller_context)	populated from headers; recorded in operational receipts	§6, §10
WB-INV-1 (api_surface enforcement)	Hub refuses unlisted verbs with 404	§3

Closing. pages was the easy half: a single document in canonical state, no build, no runtime overlay, no API. web_app is the hard half: long-running canonical transition, runtime overlay, declared API surface, full Hub routing matrix, WebSocket. Implementing P-B1 through P-B5 closes Phase 1 of stateful EasyNet's web layer and lands the agent-native web in industrial form.

The historical analogue: the early web was static documents (`easynet.page`); the modern web is applications (`easynet.web_app`). EasyNet's specification follows the same arc, deliberately. Pages first, web_apps next, each layer ratifying the protocol primitives the next layer will lean on.